Jan Ciara

+48 538 970 077 | jan.ciara@wp.pl | linkedin.com/in/jan-ciara | github.com/JanCiara

Education

Polish-Japanese Academy of Information Technology — Warsaw
Computer Science (Bachelor's Degree), Grade Average: 4.75/5.0 — *Expected Jun 2028*

Experience

Software Engineer — Sep 2026 – Present
Brown Brothers Harriman (BBH) – AI Lab — *Kraków, Poland*

Software Engineer Intern — *Mar 2026 – Aug 2026*

- Developed and maintained a shared Jenkins CI/CD library (Groovy) used by 30 Java and Python microservices; replaced **28** per-application deployment scripts with one config-driven job promoting tested container images to production via Skopeo and OpenShift.
- Built end-to-end inference logging for two production ML models (Python), streaming inputs and predictions through Kafka into an Oracle store; diagnosed an index-misalignment defect that was silently corrupting every logged record.
- Found a flaw in the shared pipeline that let services with failing test suites build and deploy to production; hard-failed the test stage, surfacing broken or missing coverage in 4 production services.
- Integrated SAST scanning (HCL AppScan on Cloud) into the shared pipeline across the team's microservices; traced empty scan results to a missing source-only mode and removed false build failures from non-actionable findings.
- Reworked a REST endpoint and its JPA associations in an internal Spring Boot skills-tracking service to accept multiple entity links per request (previously a single ID), fixing duplicate handling that silently dropped existing rows.

Helpdesk Intern — Aug 2025 – Oct 2025
Techtronic Industries (TTI) — *Warsaw, Poland*

- Deployed and configured **800+** devices in a company-wide hardware refresh (Microsoft Intune, ManageEngine).

Projects

LSM-Tree Storage Engine | *Java 21, Gradle, JUnit 5, JMH* | GitHub

- Built a persistent key-value store from scratch on an LSM-tree architecture (write-ahead log, memtable, immutable SSTables, streaming k-way merge compaction) with no external database libraries.
- Designed a custom binary SSTable format with varint (LEB128) encoding, block indexing, and a hand-written Bloom filter at 10 bits/key, so a point read touches a single 4 KiB block instead of scanning the file and a miss stays at ∼0.5 µs even across 16 tables.
- Engineered crash safety without a MANIFEST file: atomic file replacement (tmp + fsync + rename) and a deletion ordering that guarantees an interrupted compaction cannot resurrect a deleted key, with the invariant locked down by a regression test.
- Profiled the read path with JMH and found that opening the file on every query accounted for 99% of read time; a persistent channel with positional reads cut an SSTable hit from 85 µs to 6 µs (**13x**).

Chess Engine | *C++20, CMake* | GitHub

- Built a UCI-compliant chess engine searching **3.8M** nodes/second single-threaded, using bitboards, magic bitboards for sliding-piece attacks, and C++20 `<bit>` operations (popcount, count-trailing-zeros); move generation verified by perft to depth 6.
- Implemented an iterative-deepening negamax search with aspiration windows, a Zobrist-keyed transposition table, null-move pruning, late move reductions, and a quiescence search pruned by static exchange evaluation; scores **87–13–0** over 100 games against Stockfish capped at 1800 Elo.
- Optimized board representation by relocating repetition history out of the hot `Board` struct, shrinking it from 4,256 to 152 bytes and yielding a **∼4–5x** search speedup.

Skills

Languages: Java, Python, C++, SQL, Groovy, Bash
Backend & Testing: Spring Boot, Spring Data JPA, JUnit 5, Mockito, JMH
Data & Infrastructure: Oracle, Kafka, Docker, Jenkins, OpenShift, CI/CD, Gradle, Git, Linux